CSE224 Fall25 Project: Verilog Implementation of VSCPU

Ahmet Yiğit PAÇAL

Abstract

The object of this project is to design, implement, verify and synthesize a fully functional Very Simple CPU (VSCPU) using Verilog HDL. The design will support the standard VSCPU instruction set architecture (ISA) and will be extended with two new instructions: SUB and SUBi.

The EDAPLAYGROUND link of the project: https://edaplayground.com/x/RD59

Full Datapath and Control Signal Explanation

The design has 5 states: Fetch, Decode, Load R1, Load R2 and Execute states. Below given explanations for each state:

State 0: Fetch

- Purpose: Retrieve the instruction from memory.
- Operation: Place the current value of PC into memory address bus (`addr = PC`).
- Next State: Transition to Decode state.

State 1: Decode

- Purpose: To capture the fetched instruction and prepare for operand fetching.
- Operation: Load the 32-bit instruction from the data bus into the IW register (`IWN = data_fromRAM`). Load the address that will be loaded into R1 (`addr = data_fromRAM[27:14] // Different for CP, CPi and CPI`).
- Next State: Transition to Load R1/R2 state.

State 2: Load R1/R2

- Purpose: To load the first source operand into the internal register R1 and/or R2.
- Operation: The data arriving from memory is latched into the R1 register (`R1N = data_fromRAM`). If operation is immediate (except for CPIi), load the R2 register with immediate value (`R2N = IW[13:0]`), else load the address that will be loaded into R2 (`addr = IW[13:0]`).
- Next State: Transition to Load R2 state if operation is non-immediate, if it is skip Load R2 state and transition to Execute state.

State 3: Load R2

- Purpose: To load the first source operand into the internal register R2.
- Operation: The data arriving from memory is latched into the R2 register (`R2N = data_fromRAM`).
- Next State: Transition to Execute state.

State 4: Execute

- Purpose: To perform the ALU operation and write the result back to memory.
- Operation: The ALU performs the specific operation (ADD, NAND, MUL, etc.) based on the opcode (`data_toRAM = ... // Corresponding ALU operation`). Write enable is set to 1 (`wrEn = 1'b1 // Except for BZJ and BZJi`), and the result is written to RAM . The address bus is set to the destination (`addr = IW[27:14] // Except for CPIi`). Program Counter is incremented (`PCN = PC + 14'd1`) or updated to a new target for branch (BZJ) instructions
- Next State: Transition back to Fetch state to begin a new instruction cycle.

Implementation of Instructions

Registers

```
reg [31:0] IW; // Instruction Word
reg [13:0] PC; // Program Counter
reg [31:0] R1; // General-Purpose Register
reg [31:0] R2; // General-Purpose Register
```

Instruction Word

Each instruction has a fixed length of 32.

```
IW[31:29]  // opcode
IW[28]     // im
IW[27:14]  // A
IW[13:0]   // B
```

Implementations

- ADD (opcode: 000, im: 0)

```
R1          <- mem[ A ]
R2          <- mem[ B ]
mem[ A ]    <- (R1 + R2)
PC          <- PC + 1
```

- ADDi (opcode: 000, im: 1)

```
R1          <- mem[ A ]
R2          <- B
mem[ A ]    <- (R1 + R2)
PC          <- PC + 1
```

- NAND (opcode: 001, im: 0)

```
R1          <- mem[ A ]
R2          <- mem[ B ]
mem[ A ]    <- ~(R1 & R2)
PC          <- PC + 1
```

- NANDi (opcode: 001, im: 1)

```
R1          <- mem[ A ]
R2          <- B
mem[ A ]    <- ~(R1 & R2)
PC          <- PC + 1
```

- SRL (opcode: 010, im: 0)

```
R1          <- mem[ A ]
R2          <- mem[ B ]
mem[ A ]    <- (R2 < 32) ? (R1 >> R2) : (R1 << (R2-32))
PC          <- PC + 1
```

- SRLi (opcode: 010, im: 1)

```
R1          <- mem[ A ]
R2          <- B
mem[ A ]    <- (R1 + R2)
PC          <- PC + 1
```

- LT (opcode: 011, im: 0)

```
R1          <- mem[ A ]
R2          <- mem[ B ]
mem[ A ]    <- (R1 < R2) ? 1 : 0
PC          <- PC + 1
```

- LTi (opcode: 011, im: 1)

```
R1          <- mem[ A ]
R2          <- B
mem[ A ]    <- (R1 < R2) ? 1 : 0
PC          <- PC + 1
```

- CP (opcode: 100, im: 0)

```
R2          <- mem [ B ]
mem[ A ]    <- R2
PC          <- PC + 1
```

- CPi (opcode: 100, im: 1)

```
R2          <- B
```

```
mem[ A ]   <- R2
PC         <- PC + 1
```

- CPI (opcode: 101, im: 0)

```
R1         <- mem[ B ]
R2         <- mem[ R1 ]
mem[ R1 ]  <- R2
PC         <- PC + 1
```

- CPIi (opcode: 101, im: 1)

```
R1         <- mem[ A ]
R2         <- mem[ B ]
mem[ A ]   <- (R1 + R2)
PC         <- PC + 1
```

- BZJ (opcode: 110, im: 0)

```
R1         <- mem[ A ]
R2         <- mem[ B ]
PC         <- (R2 == 0) ? R1 : (PC + 1)
```

- BZJi (opcode: 110, im: 1)

```
R1         <- mem[ A ]
R2         <- B
PC         <- (R2 == 0) ? R1 : (PC + 1)
```

- MUL (opcode: 111, im: 0)

```
R1         <- mem[ A ]
R2         <- mem[ B ]
mem[ A ]   <- (R1 * R2)
PC         <- PC + 1
```

- MULi (opcode: 111, im: 1)

```
R1         <- mem[ A ]
R2         <- B
mem[ A ]   <- (R1 * R2)
PC         <- PC + 1
```

Extended Instructions

- SUB (opcode: 000, im: 0, IW[13] = 1)

```
R1         <- mem[ A ]
R2         <- mem[ B ]
mem[ A ]   <- (R1 - (~R2 + 1))
PC         <- PC + 1
```

- SUBi (opcode: 000, im: 1, IW[13] = 1)

```
R1         <- mem[ A ]
R2         <- B
```

```
mem[ A ]   <- (R1 - (~R2 + 1))
PC         <- PC + 1
```

Simulation and Verification

After loading the given assembly program into ram in testbench file (*VSCPU_tb.v*), below are the verifications for each instruction:

- ADD

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: ADD     100   101
    next program counter     :     2
    Memory content before executing instruction
    mem[         100 ]          :          5
    mem[         101 ]          :          8
    Memory content after executing instruction
    mem[         100 ]          :         13
    mem[         101 ]          :          8
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- ADDi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: ADDi    104     5
    next program counter     :     6
    Memory content before executing instruction
    mem[         104 ]          :        208
    Memory content after executing instruction
    mem[         104 ]          :        213
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- NAND

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: NAND    104   108
    next program counter     :     7
    Memory content before executing instruction
    mem[         104 ]          :        213
    mem[         108 ]          :      65543
    Memory content after executing instruction
    mem[         104 ]          : 4294967290
    mem[         108 ]          :      65543
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- NANDi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: NANDi   104     5
    next program counter     :     8
    Memory content before executing instruction
    mem[         104 ]          : 4294967290
    Memory content after executing instruction
    mem[         104 ]          : 4294967295
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- SRL

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: SRL     108   102
    next program counter     :     9
    Memory content before executing instruction
    mem[         108 ]          :      65543
    mem[         102 ]          :          8
    Memory content after executing instruction
    mem[         108 ]          :        256
    mem[         102 ]          :          8
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- SRLi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: SRLi    102     1
    next program counter     :     4
    Memory content before executing instruction
    mem[         102 ]          :         16
    Memory content after executing instruction
    mem[         102 ]          :          8
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- LT

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: LT      112   111
    next program counter     :    13
    Memory content before executing instruction
    mem[         112 ]          :          2
    mem[         111 ]          :          1
    Memory content after executing instruction
    mem[         112 ]          :          0
    mem[         111 ]          :          1
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- LTi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: LTi     105     2
    next program counter     :    22
    Memory content before executing instruction
    mem[         105 ]          :          2
    Memory content after executing instruction
    mem[         105 ]          :          0
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- CP

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: CP      104   100
    next program counter     :     5
    Memory content before executing instruction
    mem[         104 ]          :          x
    mem[         100 ]          :        208
    Memory content after executing instruction
    mem[         104 ]          :        208
    mem[         100 ]          :        208
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- CPi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: CPi     110     3
    next program counter     :     1
    Memory content before executing instruction
    mem[         110 ]          :          x
    Memory content after executing instruction
    mem[         110 ]          :          3
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- CPI

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: CPI     121   102
    next program counter     :    56
    Memory content before executing instruction
    mem[         121 ]          :          x
    mem[         102 ]          :          2
    mem[          2 ]          : 3759734886
    Memory content after executing instruction
    mem[         121 ]          : 3759734886
    mem[         102 ]          :          2
    mem[          2 ]          : 3759734886
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- CPIi

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: CPIi    114   111
    next program counter     :    55
    Memory content before executing instruction
    mem[         114 ]          :        120
    mem[         111 ]          :          1
    mem[        120 ]          :          x
    Memory content after executing instruction
    mem[         114 ]          :        120
    mem[         111 ]          :          1
    mem[        120 ]          :          1
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- BZJ

```
* * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: BZJ     111   112
    next program counter     :     1
    Memory content before executing instruction
    mem[         111 ]          :          1
    mem[         112 ]          :          0
    Memory content after executing instruction
    mem[         111 ]          :          1
    mem[         112 ]          :          0
* * * * * * * * * * * * * * * * * * * * * * * * *
```

- BZJi

```
* * * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: BZJi     101     11
    next program counter      :     19
    Memory content before executing instruction
    mem[          101 ]           :           8
    Memory content after executing instruction
    mem[          101 ]           :           8
* * * * * * * * * * * * * * * * * * * * * * * * * *
```

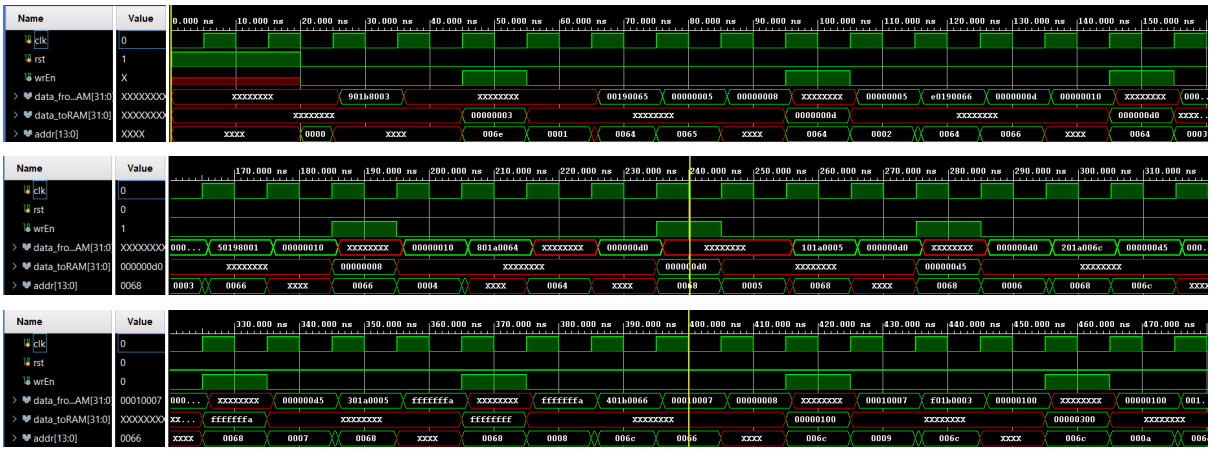
- MUL

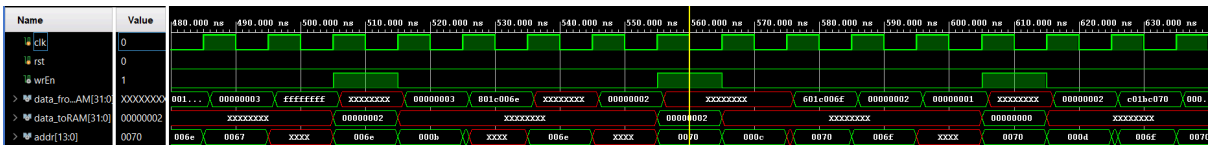
```
* * * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: MUL      100    102
    next program counter      :      3
    Memory content before executing instruction
    mem[          100 ]           :          13
    mem[          102 ]           :          16
    Memory content after executing instruction
    mem[          100 ]           :         208
    mem[          102 ]           :          16
* * * * * * * * * * * * * * * * * * * * * * * * * *
```

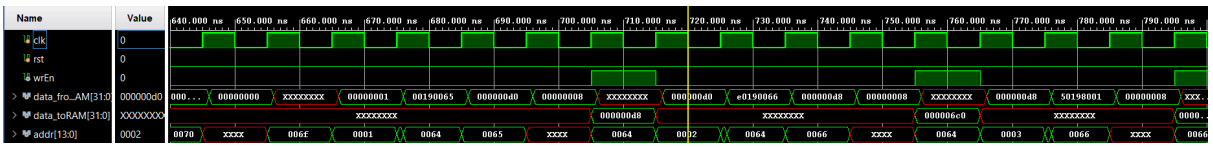
- MULi

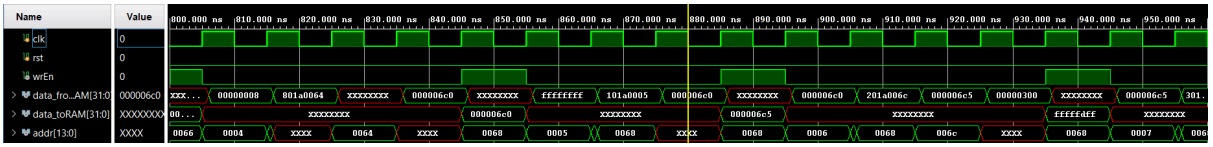
```
* * * * * * * * * * * * * * * * * * * * * * * * * *
    current_instruction: MULi     108      3
    next program counter      :     10
    Memory content before executing instruction
    mem[          108 ]           :         256
    Memory content after executing instruction
    mem[          108 ]           :         768
* * * * * * * * * * * * * * * * * * * * * * * * * *
```

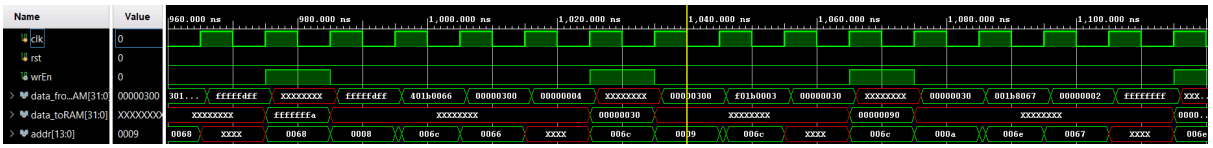
The Waveform of given program:

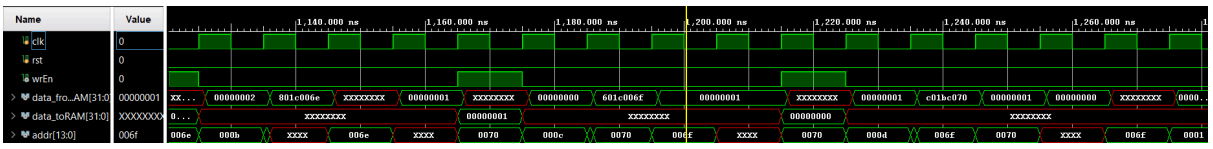


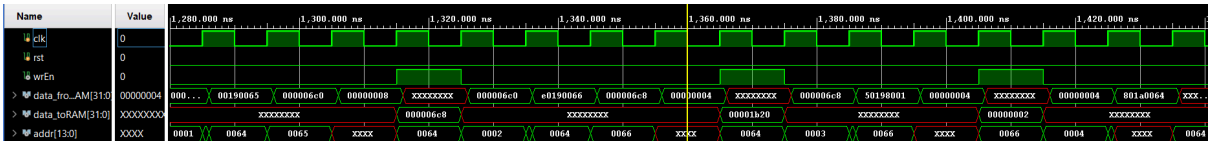

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

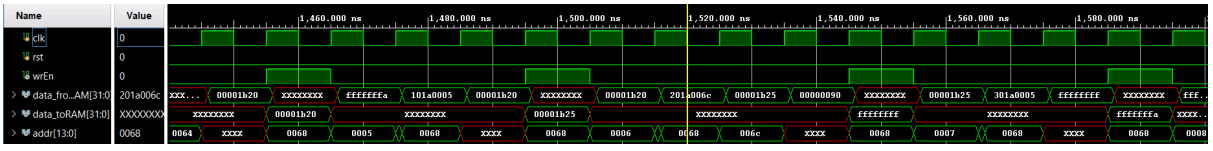

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

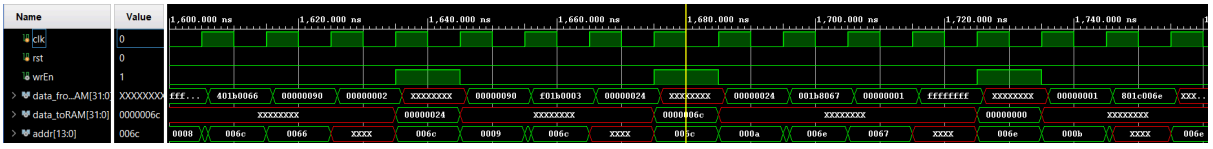

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

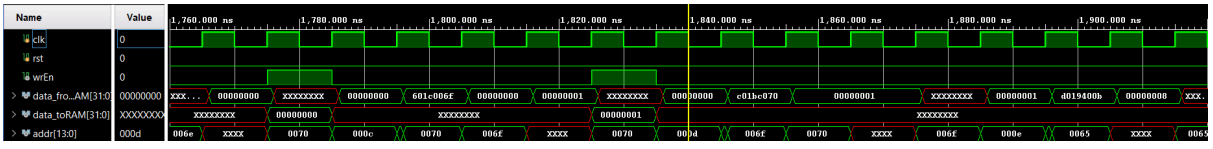

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

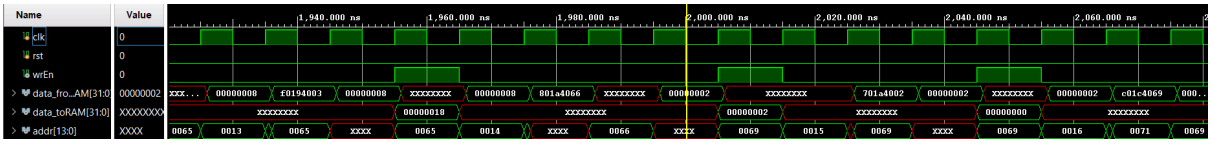

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

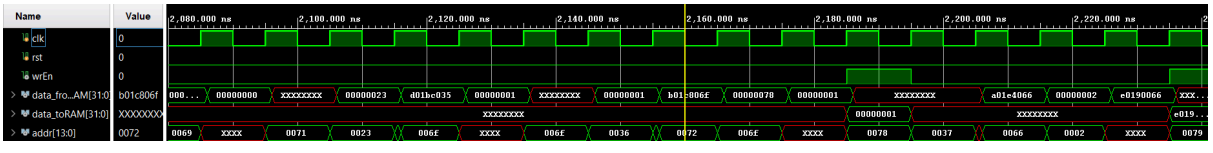

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]


Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]


Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]


Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]


Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]


Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

Name
Value
clk
rst
wrEn
data_fro...AM[31:0]
data_toRAM[31:0]
addr[13:0]

Synthesis Results

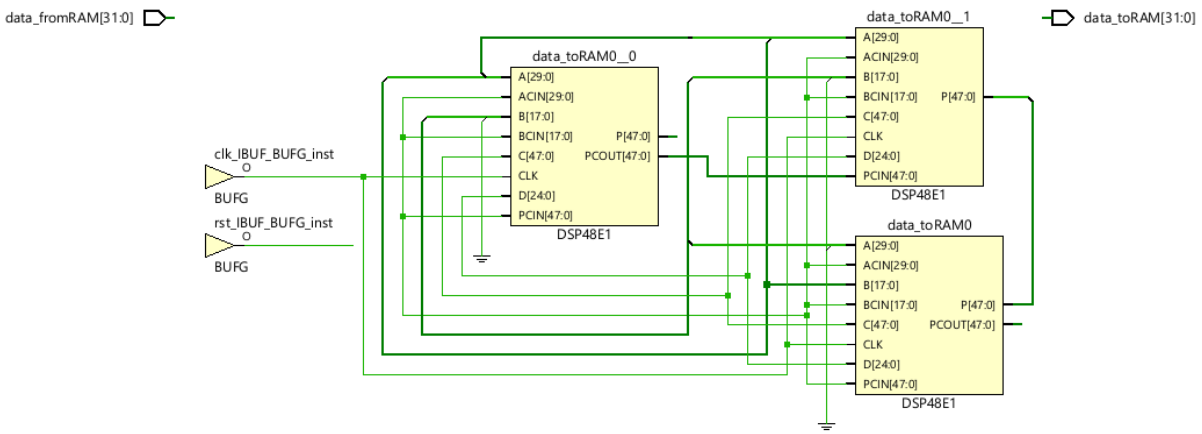
Resource	Utilization	Available	Utilization%
LUT	570	41000	1.39
FF	256	82000	0.31
DSP	3	240	1.25
IO	81	300	27.00

In the constraint file (*clock.xdc*), sys_clk is created with period 10000 ns (100000 MHz frequency) with line below:

```
create_clock -period 10.000 -name sys_clk [get_ports clk]
```

Worst Negative Slack (WNS)	Worst Hold Slack (WHS)	Worst Pulse Width Slack (WPWS)
6.828 ns	0.185 ns	4.650 ns

Dataflow Design



Ahmet Yiğit PAÇAL
20230702005